

Código de Golay extendido (24, 12) — Parte B

Modelo de referencia en software · Trabajo Práctico 2 · Curso de FEC 2026

Villar, Pedro

Qué se entrega

El paquete `golay/`, construido sobre `fec_algebra`, la librería de campos de Galois del Trabajo Práctico

1. Es el *golden model* contra el cual se verifica el RTL de la Parte C.

módulo	ejercicio	contenido
<code>bits.py</code>	4	la convención de bits del enunciado
<code>linalg.py</code>	4	álgebra matricial sobre $GF(2^m)$
<code>matrices.py</code>	5	$B, G = [I \mid B], H = [B \mid I]$
<code>encoder.py</code>	5, 6	codificador matricial y enumeración
<code>decoder.py</code>	6	algoritmo de cuatro casos
<code>cosets.py</code>	6	tabla de síndromes derivada de H
<code>stimulus.py</code>	7b	estímulo para los testbenches de la Parte C

Todos los números de este informe los produce `gen_parte_b.py` corriendo el modelo, y el script verifica cada uno contra las tablas del enunciado con `assert` antes de escribirlo.

1 Ejercicio 4 — De $GF(2^m)$ a $GF(2)$

1.1 El campo con $m = 1$

Se instancia la clase del Trabajo Práctico 1 con $m = 1$ y $P(x) = x + 1$:

`GF(m=1, primitive_poly=0b1)`

El campo queda con $2^1 = 2$ elementos y sus tablas se reducen a las de $GF(2)$:

$+$	$ $	0	1	$\cdot$	$ $	0	1	a	$ $	a^{-1}
0	$ $	0	1	0	$ $	0	0	0	$ $	—
1	$ $	1	0	1	$ $	0	1	1	$ $	1

La suma es XOR y el producto es AND, como tiene que ser. El 0 no tiene inverso y la librería levanta `GFZeroDivisionError`.

1.2 ¿Hizo falta corregir la librería?

No. Ninguna parte de la implementación asume $m \geq 2$. Vale la pena mostrar por qué, porque en dos lugares la fórmula general degenera y sigue dando bien.

Producto. El lazo de `multiply` recorre $m = 1$ iteración:

$$\text{result} = \begin{cases} a & \text{si } b \& 1 \\ 0 & \text{si no} \end{cases} \implies a \cdot b = a \wedge b.$$

La rama de reducción módulo el primitivo llega a ejecutarse, pero opera sobre x después de que ya se usó, y el enmascarado final la vuelve irrelevante. El resultado es el producto de $GF(2)$.

Inverso. La librería calcula $a^{-1} = a^{2^m-2}$, que para $m = 1$ da $a^0 = 1$ para cualquier a . Parece un caso degenerado y sin embargo es correcto, justamente porque el único elemento invertible de $GF(2)$ es el 1 y su inverso es 1. El $a = 0$ se ataja antes con una excepción.

El test `test_no_hizo_falta_tocar_la_libreria` deja esto fijado: si alguien más adelante “optimiza” `fec_algebra` asumiendo $m \geq 2$, la suite lo detecta.

1.3 El álgebra matricial agregada

El Trabajo Práctico 1 dejó los tipos (`GF`, `GFElement`, `GFPoly`) pero no el álgebra de matrices. Es lo que agrega `golay/linalg.py`, como funciones puras sobre listas de `GFElement`, genéricas sobre cualquier GF y no solo sobre $GF(2)$:

operación	función
producto vector-matriz, $v \cdot M$	<code>vec_mat(v, M)</code>
producto matriz-matriz, $A \cdot B$	<code>mat_mat(A, B)</code>
peso de Hamming, $w(v)$	<code>weight(v)</code>
distancia de Hamming, $w(u - v)$	<code>distance(u, v)</code>
suma de vectores y de matrices	<code>vec_add, mat_add</code>
traspuesta, identidad, concatenación	<code>transpose, identity, hstack</code>

Un detalle de `vec_mat` que conecta con la Parte A: se saltan los términos con $v_i = 0$, que en $GF(2)$ son la mayoría, así que el producto se vuelve la suma de las filas seleccionadas por v ,

$$v \cdot M = \bigoplus_{i: v_i=1} M_i,$$

que es exactamente como se hace la cuenta a mano.

La convención de bits del enunciado ($m[11 - i]$ es m_i) vive en `bits.py` y en ningún otro lado. Un desajuste de convención no rompe nada ruidosamente: devuelve números plausibles y equivocados.

2 Ejercicio 5 — Construcción y verificación

2.1 G y H con las operaciones nuevas

Las doce constantes de B son el único dato duro del modelo. Todo lo demás sale de `linalg`:

B	<code>to_vec</code> de cada fila del enunciado
G	<code>hstack(identity(12), B)</code>
H	<code>hstack(B, identity(12))</code>
H^T	<code>transpose(H)</code>

2.2 Las tres propiedades del Ejercicio 1

Verificadas con los productos matriciales, no a mano:

propiedad	cómo se verifica	resultado
$B^T = B$	<code>equal(B, transpose(B))</code>	verdadero
$B^2 = I$	<code>equal(mat_mat(B,B), identity(12))</code>	verdadero
$G H^T = 0$	<code>is_zero(mat_mat(G, transpose(H)))</code>	verdadero

Coinciden con las cuentas a mano de la Parte A.

2.3 Distribución de pesos por fuerza bruta

Se enumeran las $2^{12} = 4096$ palabras código aplicando $v = m \cdot G$ a cada mensaje, y se cuenta el peso de cada una:

peso w	0	8	12	16	24	total
modelo (fuerza bruta)	1	759	2576	759	1	4096
enunciado	1	759	2576	759	1	4096

Coincide exactamente con la tabla del Ejercicio 2. De ahí,

$$d_{\min} = \min\{w(v) : v \in C, v \neq 0\} = 8.$$

3 Ejercicio 6 — Codificador y decodificador

3.1 Codificador

Matricial y sin atajos: $v = m \cdot G$ con `vec_mat`. Como $G = [I \mid B]$ el resultado es el mensaje seguido de su paridad, pero se calcula con el producto completo para que el modelo sea el que pide el enunciado y no una equivalencia. Verificado contra la Parte A: $m = \text{A5C}$ da $v = \text{A5C9A5}$.

3.2 Decodificador de cuatro casos

Las funciones del decodificador están a propósito en correspondencia uno a uno con los submódulos del RTL, con los mismos nombres y las mismas entradas y salidas:

RTL (Parte C)	referencia	etapa
<code>golay_syndrome</code>	<code>syndrome(rx)</code>	E1
<code>golay_mult_b</code>	<code>mult_b(vec)</code>	E2
<code>popcount12</code>	<code>popcount12(vec)</code>	E2
<code>golay_row_search</code>	<code>row_search(vec)</code>	E2
<code>golay_err_gen</code>	<code>err_gen(...)</code>	E3
<code>golay_correct</code>	<code>correct(rx, err)</code>	E3

`decode()` no reimplementa nada: compone esas seis en el mismo orden que el pipeline de tres etapas. Así el testbench de cada submódulo compara contra su función homónima y el de integración contra `decode()`.

Dos decisiones que salieron de leer las interfaces obligatorias del enunciado:

o_corrected. El módulo `golay_correct` recibe únicamente `i_rx` e `i_err`. Entonces su salida no puede ser otra cosa que `i_err`: hubo corrección si y solo si el patrón de error es no nulo.

o_idx y o_res sin solución. Cuando `golay_row_search` no encuentra ninguna fila, esos dos puertos son *don't-care*. El modelo devuelve cero y el testbench no los debe comparar en ese caso.

3.3 Tabla de síndromes derivada de H

El algoritmo de cuatro casos no necesita ninguna tabla: resuelve por pesos y búsqueda sobre las filas b_i . Así que la tabla que pide el enunciado se usa para otra cosa, que es más útil: como **decodificador independiente** que sirve de oráculo, sin compartir una sola línea de lógica con el de cuatro casos.

Se construye recorriendo todos los patrones de error de peso ≤ 4 , calculando el síndrome de cada uno y agrupándolos. Para cada síndrome se guardan *todos* los patrones del peso mínimo en que aparece, que es lo que después permite distinguir un coset corregible de uno ambiguo:

peso del líder	líderes mínimos	cosets	
0	1	1	corregible
1	1	24	corregible
2	1	276	corregible
3	1	2024	corregible
4	6	1771	ambiguo, solo detectable
		4096	$= 2^{12}$

Los 2325 cosets de peso ≤ 3 tienen líder único: esos patrones se corrigen. Los 1771 restantes tienen seis líderes de peso 4 empatados, y sin forma de elegir entre ellos lo único que se puede hacer es declarar la palabra no corregible.

Por qué exactamente seis. Si e y e' son dos líderes del mismo coset, su suma $e + e'$ es una palabra código. Como $w(e) = w(e') = 4$, esa suma pesa a lo sumo 8; y como es no nula y $d_{\min} = 8$, pesa exactamente 8. Eso solo puede pasar si los soportes de e y e' son *disjuntos*. Seis bloques disjuntos de cuatro posiciones cubren las 24, y ahí se termina: no entra un séptimo.

líder	soporte
e_1	1000 0000 0000 1001 1000 0000
e_2	0101 0000 0010 0000 0001 0000
e_3	0010 1000 0001 0000 0010 0000
e_4	0000 0111 0000 0000 0100 0000
e_5	0000 0000 1100 0110 0000 0000
e_6	0000 0000 0000 0000 0000 1111
$\vee$	1111 1111 1111 1111 1111 1111

El script verifica que los 1771 cosets ambiguos tienen esta estructura, no solo el del ejemplo.

4 Ejercicio 7 — Caracterización del decodificador

4.1 Barrido de los patrones de peso ≤ 4

Para cada peso w entre 0 y 4 se recorre la totalidad de los patrones de error de ese peso y se clasifica el resultado en tres categorías:

peso de e	0	1	2	3	4
patrones	1	24	276	2024	10626
corregidos de forma exacta	1	24	276	2024	0
decodificados a otra palabra	0	0	0	0	0
detectados como no corregibles	0	0	0	0	10626

Coincide exactamente con la tabla del enunciado. La fila de “decodificados a otra palabra” da cero en todas las columnas: el decodificador nunca entrega una palabra equivocada en silencio, o corrige bien o levanta la bandera.

El barrido se hace sobre la palabra nula. Vale para cualquier palabra transmitida porque el síndrome depende solo de e (ver §5), así que decodificar e es equivalente a decodificar $v + e$ para cualquier v del código.

Esta tabla no es un dato empírico: es la estructura de cosets de la sección anterior leída de otra manera. Los $1 + 24 + 276 + 2024$ patrones de peso ≤ 3 son justo los líderes únicos, y los 10626 de peso 4 son los 1771×6 líderes empatados.

4.2 Estímulo para la Parte C

No se emiten archivos de vectores. Siguiendo el patrón de la cátedra en la Unidad 3, los testbenches de cocotb importan el modelo por `PYTHONPATH` y generan el estímulo adentro del propio test.

Formato. Cada generador de `stimulus.py` produce diccionarios cuyas claves son exactamente los nombres de los puertos del módulo RTL correspondiente. Las claves que empiezan con `_` son metadatos para el reporte y no son puertos. Un testbench se escribe entonces sin traducir nada:

```

from golay.stimulus import mult_b_vectors

for vec in mult_b_vectors():
    dut.i_vec.value = vec[i_vec]
    await Timer(2, "ns")
    assert int(dut.o_vec.value) == vec[o_vec]

```

Cobertura. Los submódulos de 12 bits se barren exhaustivamente, que es lo que pide el Ejercicio 10. El decodificador completo no se puede barrer: hay $2^{24} = 16,7$ millones de palabras recibidas posibles. Para ese se cura un conjunto.

módulo	generador	casos	criterio
golay_mult_b	mult_b_vectors	4096	barrido exhaustivo
popcount12	popcount12_vectors	4096	barrido exhaustivo
golay_row_search	row_search_vectors	4096	barrido exhaustivo
golay_encoder	encoder_vectors	4096	barrido exhaustivo
golay_syndrome	syndrome_vectors	6420	4096 palabras código + errores
golay_decoder	decoder_vectors	15826	curado, cinco ramas

El conjunto del decodificador son las 4096 palabras código sin error, más todos los patrones de peso 1, 2 y 3 aplicados a cinco palabras código distintas, más una submuestra de patrones de peso 4. Las cinco ramas del algoritmo quedan cubiertas:

rama	casos	resuelve por
1	5586	$w(s) \leq 3$
2	4740	$\exists i : w(s \oplus b_i) \leq 2$
3	1430	$w(q) \leq 3$
4	3960	$\exists i : w(q \oplus b_i) \leq 2$
5	110	no corregible

5 Las dos preguntas del enunciado

5.1 Por qué el síndrome solo depende del patrón de error

Sea $r = v + e$ con v palabra código. Por definición del síndrome y por linealidad del producto matricial:

$$s = r H^T = (v + e) H^T = \underbrace{v H^T}_{=0} + e H^T = e H^T.$$

El primer término se anula porque $v = m \cdot G$ y $G H^T = 0$, que es justamente la propiedad verificada en el Ejercicio 5: toda palabra código está en el núcleo de H^T . El mensaje transmitido desaparece de la cuenta y lo que queda es una función únicamente de e .

La consecuencia práctica es la que organiza todo el decodificador: los 2^{24} vectores recibidos se reparten en $2^{12} = 4096$ clases según su síndrome, dos vectores de la misma clase difieren en una palabra código, y el síndrome no puede distinguir entre ellos. Lo único que puede hacer el decodificador es elegir un representante de la clase, y la elección sensata es el de menor peso.

5.2 Por qué $d_{\min} = 8$ detecta cuatro errores pero no los corrige

Detecta. Si e tiene peso 4, $r = v + e$ nunca puede ser otra palabra código: si lo fuera, e sería la diferencia de dos palabras código, o sea una palabra código de peso 4, y el peso mínimo no nulo del código es 8. Entonces el síndrome es no nulo y el error siempre se nota. En general esto vale hasta peso $d_{\min} - 1 = 7$.

No corrige. Corregir exige elegir un patrón de error, y para eso las esferas de radio t alrededor de cada palabra código no se pueden tocar, lo que pide $d_{\min} \geq 2t + 1$:

$$2t + 1 \leq 8 \implies t = \left\lfloor \frac{8 - 1}{2} \right\rfloor = 3.$$

Corrigiendo $t = 3$ y detectando l errores más hace falta $d_{\min} \geq t + l + 1$, o sea $l \leq 8 - 3 - 1 = 4$: corrige 3, detecta 4.

La cuenta de arriba dice que no se puede; la estructura de cosets dice exactamente por qué. Un patrón de peso 4 cae en un coset donde hay *seis* patrones de peso 4, todos igual de plausibles y con soportes disjuntos. El síndrome no alcanza para distinguirlos, y elegir uno sería acertar una vez de cada seis. Por eso el algoritmo se detiene en el caso 5 y levanta `o_uncorrectable` en lugar de arriesgar.

Esto es lo que se ve en el Ejercicio 3 de la Parte A con $r_3 = \text{A5C9AA}$, cuyo error real pesa 4, y en la última columna de la tabla del Ejercicio 7a: los 10626 patrones de peso 4 se detectan, ninguno se corrige, y ninguno se decodifica mal.

Resumen de la Parte B

Ej. 4	$GF(m=1, P(x)=x+1)$ da $GF(2)$; no hubo que tocar la librería
Ej. 4	<code>linalg: vec_mat, mat_mat, weight</code> , genéricos sobre $GF(2^m)$
Ej. 5	$B^T = B, B^2 = I, G H^T = 0$ verificadas con productos matriciales
Ej. 5	pesos por fuerza bruta = tabla del enunciado; $d_{\min} = 8$
Ej. 6	codificador matricial, decodificador de cuatro casos, seis funciones espejo del RTL
Ej. 6	tabla de síndromes desde H : 2325 cosets corregibles, 1771 ambiguos
Ej. 7a	barrido de peso ≤ 4 = tabla del enunciado, cero decodificaciones erróneas
Ej. 7b	15826 casos para el decodificador, las cinco ramas cubiertas
